

Sarcasm Detection

Jordan Prasad
Department of
Computer Science
University of Victoria
Victoria, BC
V00885627

Tomislav Stojcic
Department of
Computer Science
University of Victoria
Victoria, BC
V01024183

Kevin Nguyen
Department of
Computer Science
University of Victoria
Victoria, BC
V01063686

Mitsuhiro Eisaki
Department of
Computer Science
University of Victoria
Victoria, BC
V01046519

1 Abstract

Detecting sarcasm in short text is a challenging task in natural language processing due to its reliance on subtle linguistic cues, contextual understanding, and incongruity between literal and intended meaning. This project investigates multiple machine learning models for sarcasm detection using a merged dataset of approximately 39,500 news headlines, satirical headlines, and Reddit comments labeled for sarcasm. Preprocessing and feature engineering were completed before implementing three baseline classifiers and two neural network architectures. The first neural network used TF-IDF and numeric features, while the second leveraged Sentence-BERT (SBERT) embeddings to capture sentence-level semantics. The models were trained and evaluated using a 60-20-20 split for training, validation, and testing. Average F1 score was used as the main evaluation metric. The best-performing model, the SBERT-based neural network, achieved a 76% F1 score. The results demonstrate that combining deep semantic embeddings with traditional features significantly improves sarcasm detection. The final section discusses limitations and proposes future directions, such as incorporating conversational context and experimenting with larger transformer models.

2 Introduction and Problem Statement

Sarcasm detection in written text is a significant challenge in natural language processing. Sarcasm involves expressing the opposite of the intended meaning, relying on subtle linguistic cues, contextual knowledge, and incongruity between literal and intended messages. Accurate detection is essential for improving the

performance of applications such as sentiment analysis, social media monitoring, and fake news detection. However, a primary challenge is the variability of sarcasm across different domains and communication styles. Models trained on a single domain often overfit the domain-specific style and contents, which significantly reduces their effectiveness on other domains [12]. Therefore, this project focuses on multi-domain sarcasm detection by combining datasets from various sources, including news headlines, satirical headlines, and Reddit comments, each exhibiting unique linguistic characteristics. To address this domain variability, the objective is to build, evaluate, and compare machine learning models that are robust across these diverse sources. Our approach integrates traditional text features, sentiment scores, and semantic sentence embeddings. This report will present a comprehensive analysis of model performance, discuss the limitations related to domain diversity, and outline future work to better capture the nuanced nature of sarcasm across contexts.

3 Related Work

Recent work on sarcasm detection increasingly leverages contextual embeddings. Khatri et al. [6] found that BERT embeddings outperformed traditional word vectors such as GloVe in capturing the nuanced semantics of sarcastic tweets. Similarly, Baruah et al. [13] showed that BERT outperformed TF-IDF-based SVM models, particularly when the conversational context was limited. These findings support our use of SBERT, which captures sentence-level semantics beyond surface-level lexical patterns.

4 Data and Methodology

4.1 Dataset

Two publicly available datasets were used for this project: the Sarcasm Headlines Dataset [1], which contains approximately 26,700 sarcastic and non-sarcastic news headlines, and the Self-Annotated Reddit Corpus (SARC) [2], comprising roughly 26,000 Reddit comments annotated for sarcasm. Before training, the datasets were preprocessed by filtering out very short samples (fewer than seven words), usernames, time stamps, vote counts, URLs, parent comments, and subreddit names. These features were removed to reduce noise and prevent the introduction of sparse or high-dimensional categorical data that could complicate model training (see Section: 6.5). After cleaning and merging the datasets, the final corpus consisted of 39,538 labeled examples, with 20,980 marked as sarcastic and 18,558 as non-sarcastic.

4.2 Feature Engineering

Three types of features were used in all models: sentiment polarity scores using TextBlob, TF-IDF vectors, and numeric linguistic cues such as word count and

punctuation counts. In addition, SBERT embeddings were added to the last model.

TF-IDF Feature Representation Term Frequency-Inverse Document Frequency (TF-IDF) reflects the importance of a word t in a document d relative to a corpus of N documents:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{|\{d' : t \in d'\}|}$$

where $\text{TF}(t, d)$ is the frequency of the term t in document d , and the denominator penalizes the common terms appearing in many documents.

Sentence-BERT (SBERT) Embeddings Sentence-BERT (SBERT) [5] is a modification of BERT [7] that produces semantically meaningful sentence embeddings. Unlike BERT, which outputs token-level vectors, SBERT applies a pooling operation over the final hidden states to produce a fixed-length sentence representation. Each sentence was embedded as a 384-dimensional vector $\mathbf{e} \in \mathbb{R}^{384}$, used as input to one of our neural classifiers.

4.3 Models and Implementation Details

Five models were implemented: three baseline classifiers and two neural networks, all trained on the combined dataset using a 60-20-20 train-validation-test split.

Baseline Classifiers: The baseline classifiers included logistic regression (implemented via `SGDClassifier` with logistic loss), linear support vector machine (via `LinearSVC`), and random forest (via `RandomForestClassifier`). These models were trained on TF-IDF vectors (10,000 features) and the numeric features described above. For logistic regression, hyperparameter tuning over the regularization strength α found the best validation performance at $\alpha = 0.0001$. The linear SVM model was tuned across several values of the inverse regularization parameter C , with $C = 0.1$ yielding the highest validation accuracy. The Random Forest classifier was trained using 100 trees with class-weight balancing to address dataset imbalance. All baseline models were implemented using the `scikit-learn` library [8].

Neural Network A: This feed-forward neural network takes the top 1,000 TF-IDF features (see Section 6.2) and numeric features as input. It consists of two hidden layers with ReLU activations: the first with 128 neurons and 0.5 dropout, followed by a second with 64 neurons and 0.3 dropout. L2 regularization ($\lambda = 0.001$) is applied to all layers. The output layer uses a sigmoid activation for binary classification.

Neural Network B: This model extends Neural Network A by incorporating 384-dimensional SBERT embeddings alongside the existing TF-IDF and numeric features, allowing it to capture both lexical patterns and deeper semantic context. The architecture and regularization remain identical to Neural Network A.

Both neural networks were trained using the Adam optimizer [4] with a learning rate of 0.001 and binary cross-entropy loss. Early stopping, L2 regularization, and dropout were added to combat overfitting. The models were implemented in TensorFlow [9] and Keras [10] with a batch size of 32.

5 Results

Model	Average F1 Score (Test Set)
Logistic Regression	69%
Support Vector Machine	70%
Random Forest	68%
Neural Network A	71%
Neural Network B	76%

Table 1: Final model performance comparison based on average F1 score.

Model	Class	Precision	Recall	F1-score	# Examples
Logistic Regression	0	0.70	0.74	0.72	4198
	1	0.69	0.65	0.67	3712
Support Vector Machine	0	0.71	0.73	0.72	4198
	1	0.68	0.67	0.67	3712
Random Forest	0	0.68	0.76	0.72	4198
	1	0.68	0.59	0.64	3712
Neural Network A	0	0.73	0.73	0.73	4198
	1	0.70	0.69	0.69	3712
Neural Network B	0	0.77	0.79	0.78	4198
	1	0.75	0.73	0.74	3712

Table 2: Class-wise Precision, Recall, F1-score, and Number of Examples for Different Models

6 Discussion

6.1 Semantic Embeddings vs. Lexical Features

TF-IDF vectors are sparse, high-dimensional features that represent word frequency. They capture lexical cues but lack contextual understanding as they

treat words independently. In contrast, SBERT embeddings produce dense, low-dimensional vectors encoding sentence-level semantics and contextual relationships, enabling models to detect nuanced sarcasm beyond surface-level patterns. This explains the superior performance of Neural Network B (see Table 1), consistent with previous work on context-aware sarcasm detection [13].

6.2 TF-IDF Dimensionality

Model	1,000 Features	5,000 Features	10,000 Features
Logistic Regression	66%	69%	69%
Support Vector Machine	66%	69%	69%
Random Forest	65%	66%	67%
Neural Network A	73%	73%	73%
Neural Network B	76%	76%	74%

Table 3: Validation F1 scores for different TF-IDF feature sizes.

The differing trends across models in Table 3 illustrate how model architecture impacts the utility of sparse lexical features. For baseline models, increasing the TF-IDF feature dimensionality leads to noticeable improvements, likely because these models depend solely on lexical cues. A larger vocabulary enables them to capture a wider range of word-level patterns relevant to sarcasm. Consequently, 10,000 TF-IDF features were used for the final baseline models.

In contrast, the neural networks, especially Neural Network B, which incorporates SBERT embeddings, do not benefit from higher TF-IDF dimensionality and even exhibit a slight performance decline. This is likely due to redundancy: the dense SBERT embeddings already encode rich contextual semantics, making large TF-IDF vectors partially redundant or noisy. Moreover, higher dimensionality increases the risk of overfitting, particularly given the dataset size. These findings suggest that compact TF-IDF representations are preferable when combined with deep semantic embeddings, whereas traditional models benefit from broader lexical coverage. Therefore, 1,000 TF-IDF features were used for the neural networks.

6.3 Numeric Features

Across the board, the models performed the same or better with the addition of numeric features than without them. The table below compares average F1 scores on the validation set for each model, both with and without numeric features:

Model	w/o Numeric Features	with Numeric Features
Logistic Regression	68%	69%
Support Vector Machine	68%	69%
Random Forest	67%	67%
Neural Network A	71%	73%
Neural Network B	75%	76%

Table 4: Average F1 scores with and without numeric features on the validation set.

6.4 Optimizer and Overfitting

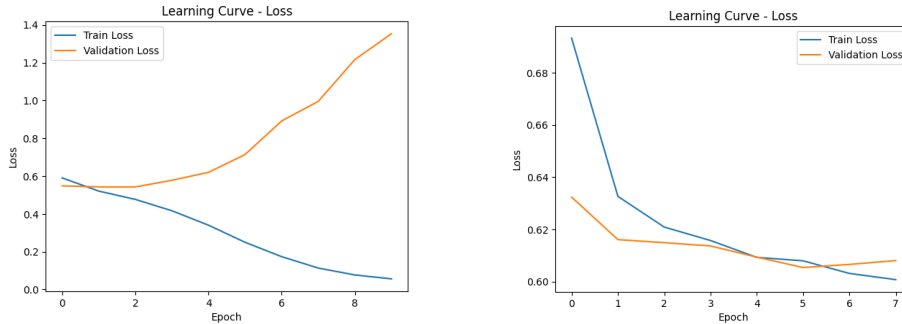


Figure 1: Neural Network A learning curves. **Left:** Before adding regularization, dropout, and early stopping. **Right:** After adding regularization, dropout, and early stopping.

Another notable observation during model development was the switch from stochastic gradient descent (SGD) to the Adam optimizer [4]. Although Adam accelerated convergence and improved training performance, it also led to greater overfitting. This aligns with prior work [3], which notes that Adam can hinder generalization without proper regularization.

In early versions of Neural Network A, a widening train-validation gap was observed after switching to Adam. To address this, L2 regularization (weight penalty $\lambda = 0.001$), dropout (50% of neurons in the first layer and 30% in the second layer) and early stopping (with patience of 2 epochs) were implemented.

As shown in Figure 1, these strategies helped mitigate overfitting, highlighting the need to balance optimizer choice with robust regularization, especially on limited or imbalanced datasets.

6.5 Limitations

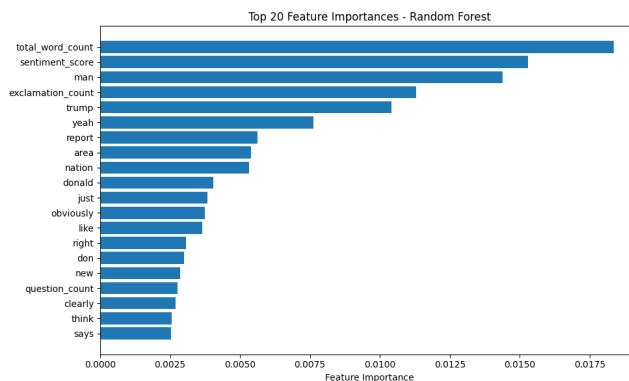


Figure 2: Random Forest feature importance

Despite reasonable performance, this project has several limitations that may hinder generalizability. First, the dataset has a slight class imbalance (53% non-sarcastic vs. 47% sarcastic), which may have skewed the results (see Table 5). Second, the models lack external context (e.g., subreddit origin or prior comments), which is often essential for interpreting sarcastic intent on Reddit.

Feature importance analysis also suggests overfitting to dataset-specific patterns. For example, the random forest model heavily relied on the word “area,” likely due to The Onion’s repeated use of headlines like “local man in area” (Figures 2 and 3). This reliance limits the model’s ability to generalize beyond the training sources.



Figure 3: Example Onion headline illustrating “area man” phrasing[11]

The dataset further reflects cultural and linguistic biases, as both Reddit and The Onion primarily represent western, English-speaking humor. Sarcasm labels in Reddit are self-annotated, adding subjectivity and potential noise. Additionally, during preprocessing, examples with fewer than seven words were excluded. Although this improved feature quality, it likely biased the dataset

toward longer, more explicit forms of sarcasm, potentially reducing model effectiveness on shorter, more subtle examples common in real-world usage.

7 Conclusion

This project explored the challenging task of detecting sarcasm across multiple domains by leveraging a merged dataset of news headlines, satirical headlines, and Reddit comments. The results demonstrate that combining deep semantic embeddings with lexical and numeric features significantly improves sarcasm detection performance, with the SBERT-based neural network achieving the highest average F1 score of 76%.

Despite these promising outcomes, the study highlights several limitations, including class imbalance, reliance on surface-level linguistic cues, and lack of conversational context. Additionally, some models exhibited potential overfitting to domain-specific stylistic patterns, which may affect generalizability.

Future work should explore the integration of richer contextual information, such as conversation threads and user metadata, as well as experimenting with larger transformer-based models and domain adaptation techniques to further enhance sarcasm detection robustness. Overall, this work contributes to the growing evidence that combining semantic embeddings with traditional features is a valuable approach for nuanced language understanding tasks like sarcasm detection.

8 References

- [1] R. Misra. *News Headlines Dataset for Sarcasm Detection*. 2023. <https://www.kaggle.com/datasets/rmisra/news-headlines-dataset-for-sarcasm-detection>
- [2] M. Khodak, N. Saunshi, and K. Vodrahalli. *A Large Self-Annotated Corpus for Sarcasm*. arXiv preprint arXiv:1704.05579, 2018. <https://arxiv.org/abs/1704.05579>
- [3] Difan Zou, Yuan Cao, Yuanzhi Li, and Quanquan Gu. *Understanding the Generalization of Adam in Learning Neural Networks with Proper Regularization*. 2019. <https://arxiv.org/pdf/2108.11371>
- [4] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. arXiv preprint arXiv:1412.6980, 2014. <https://arxiv.org/pdf/1412.6980>
- [5] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019. <https://aclanthology.org/D19-1410.pdf>
- [6] Akshay Khatri, Pranav P, and Dr. Anand Kumar M. *Sarcasm Detection in Tweets with BERT and GloVe Embeddings*. In *Proceedings of the Second Workshop on Figurative Language Processing*, pages 56–60, July 2020. <https://aclanthology.org/2020.figlang-1.7.pdf>.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2019. <https://arxiv.org/pdf/1810.04805>
- [8] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [9] Martín Abadi et al. *TensorFlow: Large-scale machine learning on heterogeneous systems*. 2015. Software available from <https://tensorflow.org>.
- [10] François Chollet. *Keras*. 2015. Software available from <https://keras.io>.
- [11] The Onion. *Area Man Accepts Burden of Being Only Person on Earth with No Patience for Idiots*. 2016. Available from: <https://theonion.com/area-man-accepts-burden-of-being-only-person-on-earth-w-1819579668/>.

- [12] Danqing Wang, Pengfei Liu, Ming Zhong, Jie Fu, Xipeng Qiu, and Xuanjing Huang. *Exploring Domain Shift in Extractive Text Summarization*. School of Computer Science, Fudan University; Mila and Polytechnique Montreal, 2020. Available from: <https://arxiv.org/abs/2004.05590>.
- [13] Arup Baruah, Kaushik Amar Das, Ferdous Ahmed Barbhuiya, and Kuntal Dey. *Context-Aware Sarcasm Detection Using BERT*. In *Proceedings of the Second Workshop on Figurative Language Processing*, pages 83–87, July 2020, Online. Association for Computational Linguistics. <https://aclanthology.org/2020.figlang-1.12/>